



BSc (Hons) in **Information Technology**
Specialized in AI
IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)  
'walls': list(self.walls)  
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the complete collection of all valid states that can exist in the environment. In this grid game, it includes every position the agent can occupy without moving outside the grid or entering a wall.

The search tree is created by a search algorithm while looking for a path from the starting position to the goal. Its nodes show the states explored through different paths. The same state may appear more than once in a search tree if it can be reached through different routes. Therefore, the state space describes the whole problem, while the search tree shows how the algorithm explores that problem.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set remembers the states that the agent has already discovered. Before adding a new state to the frontier, the algorithm checks whether that state is already reached.

This prevents the agent from repeatedly exploring the same positions and moving around in cycles. It also changes the algorithm from a tree search into a graph search. Without the reached set, DFS could repeatedly follow a loop between the same grid cells. It might waste time and memory or even fail to reach the food.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

In the simulation, BFS normally takes a short and direct route to the food. It explores the grid level by level, meaning that it checks all positions one step away before checking positions two steps away.

Every movement in this grid has the same cost of one. Therefore, the first path found by BFS has the smallest possible number of movements, making BFS optimal in this environment.

DFS behaves differently because it follows one path as deeply as possible before returning to explore another path. It may move far away from nearby food before finding the correct route. As a result, DFS can produce a longer and more winding path and is not guaranteed to find the optimal solution.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

On a large 1000×1000 grid, BFS may require a very large amount of memory because it stores all the nodes at the current search level in its frontier. Its space complexity is approximately $O(b^d)$, where b is the branching factor and d is the depth of the nearest goal. The number of stored nodes can grow very quickly.

UCS can experience a similar problem because it keeps many possible paths in its priority queue. This is why BFS and UCS may run out of memory on a massive grid.

DFS normally requires much less memory because it mainly stores the current path and the remaining alternatives. Its space complexity is approximately $O(bm)$, where m is the maximum search depth. However, this memory advantage has a trade-off: DFS may explore very deep, unnecessary paths, take a long time, and return a suboptimal solution.

In simple terms, BFS uses more memory but finds the shortest path in this equal-cost grid, while DFS uses less memory but may find a longer path.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING